

AIQG — Experiments Runner (Design)

Status: **Design / for review** — NOT implemented. Gates a change to the production gateway's routing of live customer LLM traffic; do not build without sign-off on §7 (Guardrails) and §13 (Risks). Owner: AIQG · Related: AIQG-AGENT-FLOW-ATTRIBUTION.md (identity ladder), aether-shared/data-models/aiqg/route-rule.md, aiqg-ui/docs/PERSONA-DASHBOARD-IA.md §6.

1. Problem & goal

The shipped **Experiments comparison view** (aiqg-ui /reports/experiments) compares two vendor/model configs *on traffic that already happened* — it can't tell you how config B *would* perform on config A's traffic, because B never ran it. Answering “is gpt-4o-mini good enough vs gpt-4o for our RAG workflow, at what cost/quality?” requires actually **running a fraction of live traffic on the variant and measuring it** — an online A/B test.

Goal: let an operator define an experiment (a cohort of traffic + two or more variants + a traffic split), have the gateway route the split deterministically and stickily, stamp each request with its variant, and report per-variant CLEAR / cost / latency / success — with hard guardrails and a kill switch, because this reroutes real customer traffic.

Non-goals (v1)

- Automatic winner selection / auto-ramp (Phase 3).
 - Full sequential/Bayesian statistics (v1 ships sample counts + a simple confidence note; see §6).
 - Prompt/parameter experiments beyond vendor/model/policy-bundle (later).
 - Multi-armed bandit allocation (later; v1 is fixed-weight split).
-

2. Prior art in TAS (what we build on)

- **Route rules** (route-rule.md, resolved in internal/policy/resolver.go) — bind a **policy bundle** to a request shape via matchers (url_path, source_app, vendor, model, customer_header), priority-ordered, first-match-wins, with a staged mode (draft → dry_run → enforce → disabled). Experiments reuse the **matcher shape** (the cohort) and the **staged-rollout discipline**, but target the *routing decision*, not a bundle.
 - **Router** (internal/routing/router.go Route() → determineStrategy() → routeByStrategy()) — owns vendor/model selection. The experiment resolver produces a **variant override** the router honors instead of its default strategy for assigned requests.
 - **Identity ladder** (AIQG-AGENT-FLOW-ATTRIBUTION.md, shipped) — gives a stable per-request assignment key: baggage user.id → conversation.id → flow_id → principal (token) → client_ip. **This is the keystone:** sticky assignment requires a stable identity, which we now have.
 - **Events pipeline** (pkg/aiqg/events) — already promotes vendor, model, workflow, CLEAR scores, cost, status, identity to Loki. We add experiment_id + variant the same way; measurement is then the existing per-dimension aggregation grouped by variant.
 - **Comparison view** (shipped) — becomes the *results* surface, fed by the live per-variant data instead of hand-picked vendor rows.
-

3. Core model

Experiment

id, tenant_id, name, description, status
cohort – matcher set (which requests are eligible)
variants[] – { key, weight, override } (key "control" = baseline routing)
assignment – { key_source, salt } (how a request maps to a variant)
guardrails – { max_traffic_pct, stop_on, min_samples } (§7)
schedule – { starts_at, ends_at } (optional)
created_by, created_at, transitions[] (audit)

Variant.override – a TYPED SET OF AXES; an experiment varies one (rarely more):
vendor / model route to a different provider/model
system_prompt override/inject the system prompt
prompt_template_id swap a named prompt template
policy_bundle_id A/B a policy bundle (reuses route/policy path)
gatekeeper_profile A/B Gatekeeper config – scan profile, redaction on/off, tokenization mode
params sampling params (temperature, max_tokens...)
control = empty override (baseline runs unchanged)
Assignment.mode – gateway_applied (gateway applies the override) OR
client_asserted (the app picks the arm + sends
`TAS-Experiment-Variant: <key>`; gateway only attributes/measures)
Cohort – route-rule-style matchers + workflow_type (Phase-2 matcher that Experiments needs from day one): url_path, source_app[], model[], workflow_type[], customer_header

A request is **eligible** if it matches the cohort AND the experiment is running AND within schedule. Eligible requests are assigned to exactly one variant; everyone else routes normally (untouched).

Experiment kinds (the variant axis is general — not just models)

- **Model/vendor swap** (gpt-4o vs 4o-mini) — cost/latency win vs quality *non-inferiority* (§6.2–6.3). The canonical case.
- **Prompt A/B** (system prompt or template v1 vs v2) — quality/efficacy *superiority*. Either gateway_applied (gateway injects the variant system prompt) or client_asserted (the app owns its prompts and just tags the arm).
- **Policy-bundle A/B** — enforcement/blocking impact (reuses the policy path).
- **Gatekeeper-impact** (redaction/tokenization profile A vs B, or on vs off) — *does context modification degrade response quality?* See §6.4 — this is a first-class case, not an afterthought: the product's premise is that Gatekeeper's inbound rewrites don't hurt answers, and this is how you prove (or bound) it.
- **Param sweep** (temperature, max_tokens) — quality/latency/cost tradeoff.

gateway_applied axes that **mutate the request body** (system_prompt, prompt_template, params, gatekeeper_profile) are more invasive than a routing override — see the risk in §13.

4. Assignment (deterministic, sticky)

key = first non-empty of [user_id, conversation_id, flow_id, principal_id, client_ip, request_id]
bucket = crc32(experiment_id + ":" + salt + ":" + key) % 10000 // 0.01% granularity
variant = walk cumulative variant weights until bucket < cumulative

- **Deterministic + sticky:** the same user/conversation always lands in the same variant for the life of the experiment — critical so a user isn’t bounced between models mid-conversation (jarring, and pollutes results). This is exactly why the identity ladder had to land first.
- **key_source** is configurable (default conversation so a multi-turn thread is coherent; user for cross-conversation stickiness; request for pure per-call randomization when no stable id exists).
- **Falls back down the ladder:** if no `user_id`, use `conversation_id`, etc., ending at a per-request UUID (effectively random) — so an un-instrumented caller still gets a clean split, just not sticky.
- **Salt** decorrelates assignment across experiments — so a user bucketed “low” in one isn’t systematically “low” in the next (sequential experiments, and the future domain/layer modes in §4.1). Concurrent *overlapping* experiments don’t co-occur on a request in v1 — see §4.1.
- **Bucketing respects max_traffic_pct:** only the first N% of buckets are eligible; the rest force control regardless of variant weights, so a 50/50 experiment capped at 10% traffic = 5% A / 5% B / 90% untouched.

4.1 Collision & concurrency (when >1 experiment matches a request)

Nearly every variant axis here — model, prompt, gatekeeper_profile, params — changes the **same outcome** (the response and its quality/cost/latency). So two overlapping experiments **confound** each other’s results. That rules out naive **layering** (independent concurrent experiments per request), which is only safe for genuinely orthogonal subsystems — which we mostly don’t have.

v1 rule — mutual exclusion by priority. A request is **claimed by the highest-priority running/dry_run experiment whose cohort matches**, and participates in that one only. Reuses the route-rule priority model (reserved ranges, first-match-wins) — one mental model, not two.

- **Claim-on-match, not claim-on-variant:** a request matching exp A leaves exp B’s pool *even if A assigned it to control* — otherwise A’s control and variant populations would differ (variant excluded from B, control not), confounding A. Each experiment’s control vs variant must come from the same claimed population.
- **Not a user-facing knob.** We do NOT expose a collision-strategy selector (layering/domains/priority menu) — it lets operators ship confounded results. The only exposed control is priority (the tiebreaker).

We DO detect collisions (what makes “pick one + document it” safe rather than silently broken): - *Static (create/start):* conservatively test the new cohort against active experiments — intersect source_app/model/workflow_type sets, treat regex url_path as “possibly overlaps” (bias to over-flag). On overlap → **warn + require explicit acknowledgment** (“exp X will be starved / may not reach significance”). - *Runtime:* a request matching >1 eligible experiment emits an experiment_collision metric and flags the lower-priority experiment’s results as **“possible confounding / starved.”**

Documented limitation: you cannot run two *overlapping* experiments concurrently without starving the lower-priority one — run them sequentially or make their cohorts disjoint.

Deferred (Phase 3), safest-first: (1) **mutually-exclusive domains** — partition a cohort into disjoint slices so N experiments each get a guaranteed, unbiased fraction (bounded total traffic); (2) **layering** — only for axes proven orthogonal in outcome, gated behind an explicit orthogonality assertion + interaction monitoring.

5. Gateway integration

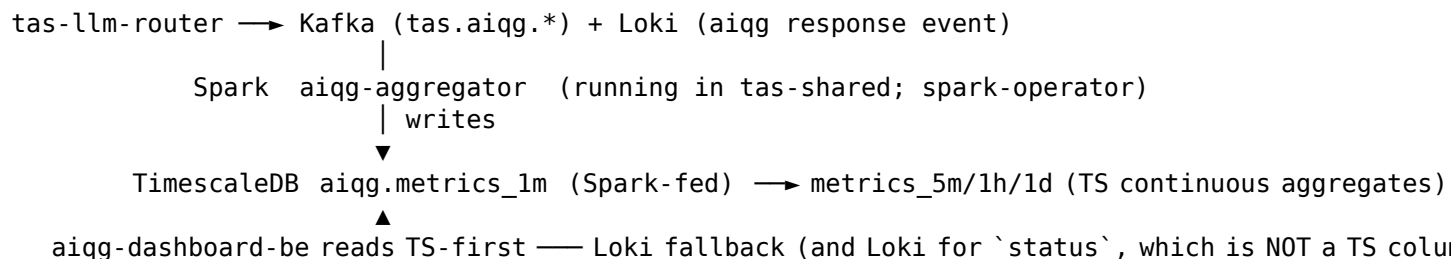
A new **experiment resolver** in `tas-llm-router`, modeled on the policy resolver (per-tenant cache, loaded from `aiqg-dashboard-be` on miss):

```
handleAIQG → resolve token → resolve policy bundle
            → resolveExperiment(ctx, req, identity) ← NEW
              match cohort → assign variant → produce *VariantDecision
            → Router.Route(...) honors VariantDecision.override
              (vendor/model) instead of its default strategy
            → events.Build stamps experiment_id + variant on the event
```

- **Override application:** `Router.Route()` gains an optional `forcedVendor/forcedModel` (from the variant) that short-circuits `determineStrategy()` for assigned requests. Control variant → no override, baseline strategy runs unchanged.
- **Header overrides / opt-out:** `TAS-Experiment`: off on a request bypasses all experiment assignment (escape hatch for a caller). A future `TAS-Experiment-Variant`: `<key>` can pin a variant for testing (dry-run only).
- **Event stamping:** add `ExperimentID` + `Variant` to the `AgentContext` sibling (or a small `ExperimentContext`); promote `experiment_id` + `variant` to the `Loki` line (same mechanism as the identity fields). No other event shape changes.
- **Fail-safe:** any resolver error, cache miss, or panic → **no assignment** (route normally). Experiments can never break routing; they can only *narrow* to control. Mirrors the policy resolver's `degrade-to-Default()`.
- **policy_bundle variant override** reuses the existing route/policy path — an experiment can also A/B a *policy bundle*, not just a model.

6. Measurement, decision metrics & what “better” means

The existing pipeline (verified live)



Rollup rows are keyed (`bucket_start`, `tenant_id`, `scope_type`, `scope_key`), `scope_type` ∈ {`account`, `workflow`, `route`, `model`, `endpoint`, `source_app`}, carrying `request_count`, **tdigest latency sketches**, `CLEAR` scores, and `cost`. One request fans out into one row per `scope_type`.

6.1 “Better” is objective-relative

There is no single “better” metric — each experiment declares a **primary objective** + **guardrails** on the rest:

- **Cost-reduction** (the common model-swap case): primary = quality **non-inferiority** (variant within a margin ϵ of control) **at lower cost**.
- **Quality-improvement** (the common prompt-A/B case): primary = quality **superiority** at acceptable cost/latency.
- **Latency/cost-only**: primary = `p95` or `$/req`, quality as a guardrail.

Non-inferiority — “*not meaningfully worse*” — is the default frame for any cheaper/faster variant, **not** “the higher number wins.”

6.2 What we can measure — objective spine + a quality layer

Objective spine (have today, cheap, exact): cost/req, tokens, p50/p95 latency, status (success/error/blocked), safety (assurance findings, NIST). Reliable; drives guardrails.

The quality gap: CLEAR **Efficacy** is MVP-computed from *structural validity + hedge/refusal signals only* (build-vs-reuse §2.12) — it does **not** measure correctness / helpfulness / completeness, and the source doc itself flags that it under-represents true quality pending LLM-as-judge. So **do not let Efficacy be the quality verdict**. An experiment whose objective is quality MUST name a real quality metric from this ladder (keyed to workflow_type):

Tier	Signal	Cost	Status
Structural	validity, parse/schema conformance, hedge/refusal; grounded- ness/citations (RAG)	free	partly today
Behavioral / implicit	regeneration & retry rate, conversation continuation vs abandonment, tool-call success, code accept / edit-distance	cheap	needs light client signals
Explicit feedback	app posts an outcome tied to response_event_id (thumbs / accepted / resolved / task reward)	cheap, gold	needs a feedback path
LLM-as-judge	a <i>different</i> judge model rubric-scores a sample (correct- ness/helpfulness/completeness), async	\$\$ + latency	the planned Phase-2 efficacy augmentation — experiments <i>consume</i> it

Decision-metric roles:

Role	Metrics (per variant vs control)	Source
Guardrails / auto-stop	error rate, p95 latency, avg cost/req, safety-finding rate	Loki (status/safety) + TS (latency/cost)
Primary verdict	the experiment’s objective metric + its chosen quality metric	per the ladder
Reported alongside	full CLEAR, tokens, sample count	TS + Loki

The additive change this requires (Spark + TS)

Per-variant reads must be cheap enough for ~per-minute guardrail polling and correct for p95 — so the **Spark aiqg-aggregator emits a new scope row**:

```
scope_type = 'experiment_variant'      (additive enum value)
scope_key = '<experiment_id>:<variant>' (emitted when the event carries experiment_id + variant)
```

Then: - **Results** = ... FROM aiqg.metrics_1m WHERE scope_type='experiment_variant' AND scope_key LIKE '<exp>:%' → per-variant CLEAR/cost/latency (p95 from the t-digest), sub-100ms. GET /api/v1/experiments/{id}/results runs this + a Loki sum by (variant, status) for the status breakdown. - **Auto-stop evaluator** — a periodic job in aiqg-dashboard-be reads the same 1m rollup + the Loki status query every ~minute, compares each variant to control, and trips the experiment to paused on a guardrail breach.

This is an **additive scope_type** (the model declares scope_type additive) plus the status-from-Loki pattern already shipped for /metrics/health and /by-vendor — not a new pipeline. It is **load-bearing for guardrails**, so it lands in Phase 2b (not deferred).

6.3 Verdict mechanics

- **Separate guardrails from the verdict.** Auto-stop runs only on the cheap/fast/unambiguous spine (error/latency/cost/safety) — never on a slow or noisy quality judge. The better/worse *verdict* may use the quality metric, sample-size-gated.
- **Non-inferiority test (default):** accept the variant if its quality metric is within margin ϵ of control (e.g. judge-score delta $\geq -2pp$) **and** it wins the objective (cost/latency). Prevents “looked cheaper, shipped a regression.”
- **Sampling:** LLM-as-judge scores a random **sample** ($\approx 5-20\%$) per variant, async, off the hot path; the judge is a *third* model (not either variant) to avoid self-preference / position bias.
- **Split-test caveat → shadow-eval:** in a live split a prompt hits only ONE arm, so same-prompt **pairwise** A/B isn’t available inline. Either score **pointwise** per arm, or run a **shadow-eval**: replay a sample of *control’s* logged requests through the variant offline and judge pairwise — zero user impact ($\sim 2\times$ cost on the sample). Shadow-eval gives the cleanest paired quality delta and pairs naturally with dry_run (§8).
- **Significance (v1):** report sample count + a coarse flag — “insufficient ($<min_samples$)”, “directional”, “clear” (non-overlapping simple CIs on the primary metric). Not a p-value engine in v1; sequential/Bayesian is Phase 3.

6.4 Gatekeeper-impact experiments

Gatekeeper modifies the **inbound** context (PII tokenization swaps values, redaction removes spans, injection-neutralization rewrites) — which *could* degrade the model’s answer. A Gatekeeper-impact experiment varies gatekeeper_profile across arms (e.g. control = current profile, variant = no-redaction / a different tokenization mode) and measures the **quality delta** with the §6.2 ladder. This directly tests the product premise that Gatekeeper’s rewrites don’t hurt answers — or quantifies the tradeoff.

- **Context-modification covariate:** alongside quality, surface *how much* Gatekeeper changed the prompt per request — count of inbound findings that triggered a redact/tokenize action, and % of prompt tokens altered (derivable from the existing assurance findings + tag actions). Lets you correlate modification *magnitude* with quality delta, not just on/off.
- **Strongest design = shadow-eval pairwise:** run the same logged prompt through both profiles offline and judge the paired responses — isolates Gatekeeper’s effect from prompt variance, with no user impact.

- A `gatekeeper_profile` variant **mutates what the model sees** (inbound request body), so it's a request-mutating axis — see §13.

6.5 Explicit feedback (gold signal) — a shared capability

Feedback is the highest-signal, lowest-bias quality metric, and useful far beyond experiments (it can also augment CLEAR Efficacy and the AI Quality report). Build it as a **standalone capability experiments consume**, not experiment-internal. (Deserves its own `aether-shared/data-models/aiqg/response-feedback.md`.)

Ingest — `POST /api/v1/feedback` (tenant-scoped; AIQG-token or Keycloak auth):

```
{ response_event_id? | client_request_id?, // correlation – one required
  signal_type, // thumb | accept_reject | rating | task_success | reward | edit_distance | custom
  value, // normalized: thumb → ±1, rating → 1..5, task_success → bool, reward → float
  source_app?, occurred_at?, metadata? }
```

Correlation, two modes: - `response_event_id` (exact) — the gateway optionally returns a `TAS-Response-Event-Id` response header (also reachable via `TAS-Trace`) so apps can capture it at call time. - `client_request_id` (app-friendly) — the app sends a stable id on the LLM call (reuses `X-Request-ID` / `baggage session.id`); the event already records it, and feedback references the same id. Resolves to the event by (tenant, id).

Storage + join — `aiqg.response_feedback(response_event_id, tenant_id, signal_type, value, source_app, occurred_at, metadata)`. At ingest, resolve the referenced event and **denormalize experiment_id + variant onto the row**, so per-variant feedback aggregation is a plain `GROUP BY` (no read-time join).

Late-arriving by nature — a thumb is immediate; “ticket resolved” may be hours/days later. A feedback-based verdict therefore **matures over time**: results windows must tolerate late feedback, and an experiment can sit in “verdict pending sufficient feedback” until enough lands.

Hygiene — tenant-scoped + rate-limited; `metadata/comments` may carry PII, so that field is optional and minimized/scanned like other payloads; feedback is never required for an experiment to run (it's the gold tier, not the floor).

6.6 LLM-as-judge rubrics, by workflow type

The judge is likewise a **shared capability** (the planned Phase-2 Efficacy augmentation); experiments consume it. The rubric is keyed to `workflow_type`, and **where a cheaper automatic signal exists, prefer it over the judge** (cheaper, unbiased):

<code>workflow_type</code>	Judge rubric (semantic)	Cheaper automatic signal — prefer when available
<code>single_turn_qa</code>	<code>correctness · helpfulness · completeness</code>	— (judge or explicit feedback)
<code>rag</code>	faithfulness to retrieved context · <code>answer relevance</code>	groundedness / citation coverage (structural, have)
<code>summarization</code>	<code>faithfulness (no hallucination) · key-point coverage · concision</code>	context-utilization / coverage proxy
<code>code_generation</code>	<code>solves the prompt · idiomatic</code>	execution: compile / run tests (beats a judge)

workflow_type	Judge rubric (semantic)	Cheaper automatic signal — prefer when available
classification_extraction	label correctness	schema conformance + label match (when labels known)
agentic	goal completion · correct tool use across turns	tool-call success + outcome feedback

- **Mode** — reference-free in production (no ground truth): **pointwise** rubric per arm on the live split, or **pairwise** (A vs B on the same prompt+context) in shadow-eval (§6.3) for a stronger paired signal.
- **Output** — per-dimension 0-1 + overall + an **abstain** option; aggregate over the sample → a per-variant judge score with a CI.
- **Bias controls (mandatory)** — judge model ≠ either variant; **randomize A/B order** in pairwise (or score both orders and average) to counter position bias; version the rubric (judge_rubric_version) so scores compare over time; periodically **calibrate the judge against a human-labeled spot-check** set.

7. Guardrails (the part that gates the build)

This reroutes **real customer LLM traffic** to a different model/vendor. Every experiment MUST carry:

1. **max_traffic_pct** (default 5%, hard cap configurable per tenant) — the ceiling on eligible traffic; the rest is forced to control.
2. **Auto-stop (stop_on)** — the gateway/aggregator trips the experiment to paused (all traffic → control) when, on ≥min_samples:
 - variant error rate > control error rate + error_delta (default +2pp), or
 - variant p95 latency > control × latency_factor (default 1.5×), or
 - variant avg cost/request > control × cost_factor (default 2×).
3. **Kill switch** — a single tenant-level “halt all experiments” flag the gateway honors on its next cache refresh (and an explicit per-experiment paused). Cache TTL bounds worst-case propagation (target ≤30s).
4. **Sticky-by-conversation default** — never bounce a user mid-thread.
5. **Dry-run mode** (§8) before any real reroute — validate cohort sizing & assignment with zero routing change.
6. **Tenant isolation** — experiments are strictly tenant-scoped; no cross-tenant cohorts (same invariant as route-rules).
7. **Audit** — every transition (start/pause/edit/stop) + auto-stops recorded in audit-log-entry, with the tripping metric.
8. **Cost ceiling (optional)** — absolute \$/day budget for the variant arm; exceed → auto-pause.

Default posture is conservative: small %, auto-stop armed, sticky, dry-run first, kill switch always available.

8. Lifecycle (staged, mirrors route-rule mode)

draft — editing; never matches traffic
dry_run — gateway ASSIGNS variants + stamps experiment_id/variant on events,

but does NOT apply the override (everyone still routes to control).
 Proves cohort sizing, assignment balance, and stickiness with ZERO routing change. The safe pre-flight.

running – live split; overrides applied within max_traffic_pct; guardrails armed
 paused – all traffic → control (manual or auto-stop); results retained
 completed – ended (schedule or operator); read-only results
 archived – audit trail only

dry_run → running is the only transition that changes production routing and should require an explicit confirm in the UI (and is audited).

9. Data model

aiqq-dashboard-be (PostgreSQL) — new tables, additive:

```
aiqq.experiment      (id, tenant_id, name, description, status,
                    cohort jsonb, assignment jsonb, guardrails jsonb,
                    starts_at, ends_at, created_by, created_at, updated_at)
aiqq.experiment_variant (id, experiment_id, key, weight, override jsonb)
aiqq.experiment_transition (id, experiment_id, from, to, actor, reason, at)
```

Gateway caches active (running/dry_run) experiments per tenant (same pattern + TTL as the policy resolver cache). New Kafka topic `tas.aiqq.experiment.changed.v1` (additive, matches the existing AIQG topic convention) invalidates the cache on edit for sub-cache-TTL propagation.

Per-variant results are served by the **scope_type='experiment_variant'** **rollup** the Spark aggregator emits (\$6) — `aiqq.metrics_lm` keyed `scope_key='<experiment_id>:<variant>'` — plus a Loki status query. No per-experiment results storage in Postgres; the metrics ride the existing Spark→TS rollup + Loki, just on a new scope dimension.

10. API (aiqq-dashboard-be)

```
GET    /api/v1/experiments      list (tenant-scoped)
POST   /api/v1/experiments      create (draft)
GET    /api/v1/experiments/{id} detail
PATCH /api/v1/experiments/{id} edit (draft/dry_run only for cohort/variants)
POST   /api/v1/experiments/{id}/transition { to: dry_run|running|paused|completed }
DELETE /api/v1/experiments/{id} archive (soft)
GET    /api/v1/experiments/{id}/results per-variant metrics + sample counts + confidence
GET    /internal/experiments?tenant=... gateway cache-load endpoint (internal auth)
```

Validation: weights sum to 100; exactly one control; max_traffic_pct ≤ tenant ceiling; cohort matchers validated like route-rules; running/paused edits restricted to guardrails + schedule (not cohort/variants — those need a new experiment to keep results clean).

11. UI (aiqq-ui — Reports ► Experiments)

Extends the existing page (today: comparison) with: - **List** of experiments (status chip, cohort summary, traffic %, variant count). - **Create/edit wizard**: cohort (reuse route-rule matcher inputs + workflow type) → variants (model/vendor/bundle + weight) → assignment key →

guardrails → schedule. dry_run → running is a guarded confirm. - **Results**: the comparison table we already built, now per *variant* (not hand-picked vendor rows), + sample counts, confidence flag, live status, and a prominent **Pause** button. Auto-stop events shown inline. - The current ad-hoc comparison stays as a “quick compare (historical)” tab.

12. Phasing

- **Phase 1 — shipped**: historical comparison view.
 - **Phase 2a — config + dry-run**: data model, CRUD API, UI wizard, gateway experiment resolver in **dry_run only** (assign + stamp, no reroute). Proves cohorts/assignment/stickiness against live traffic with zero risk.
 - **Phase 2b — live split**: enable the routing override + guardrails + auto-stop + kill switch, the `scope_type='experiment_variant'` **Spark rollup** (§6 — load-bearing for guardrails/results), and the results endpoint/UI. The gated step.
 - **Phase 3**: significance engine, auto-ramp, winner declaration, bandit allocation.
-

13. Risks & open questions

- **Production routing blast radius** — mitigated by `max_traffic_pct`, dry-run pre-flight, auto-stop, kill switch, fail-safe-to-control. Still the #1 risk; Phase 2b needs explicit sign-off.
- **Cost runaway** — a variant on a pricier model multiplies spend on its arm; cost guardrail + ceiling required.
- **Request-body-mutating axes** (`system_prompt`, `prompt_template`, `params`, `gatekeeper_profile`) are more invasive than a routing override — the gateway changes *what the model sees / how it's scanned*, not just where the call goes. Higher blast radius and harder to reason about. Mitigations: ship routing/model axes first (Phase 2b), gate mutating axes behind a separate enablement + extra dry-run scrutiny, and prefer `client_asserted` mode for prompt A/B where the app already owns its prompts. Gatekeeper-profile variants additionally interact with safety posture — never let an experiment widen the security envelope (e.g. silently disabling injection scanning) without an explicit, audited security sign-off.
- **Quality measurement is the soft spot** — CLEAR Efficacy is structural-only (§6.2); a quality verdict needs the judge/feedback layer, which adds cost, latency, and judge bias. v1 leans on non-inferiority + sampling + shadow-eval and is explicit about confidence; over-trusting Efficacy as “quality” is the trap to avoid.
- **Result validity** — confounding (time-of-day, cohort skew), insufficient samples; v1 is honest about sample size and avoids over-claiming significance.
- **Stickiness vs. coverage** — keying on conversation is coherent but a few heavy users can dominate a variant; surface per-variant unique-key counts so skew is visible.
- **Experiment collision / starvation** — overlapping experiments confound (our axes aren't orthogonal), so v1 is mutual-exclusion-by-priority: overlapping experiments can't run truly concurrently, and a low-priority one is starved. Mitigated by collision detection (warn + runtime flag) and the documented sequential/disjoint workaround. See §4.1; domains/layering deferred.
- **Open: where the override lives** — extend `Router.Route()` with a forced vendor/model arg (cleanest) vs. a synthetic high-priority route rule. Leaning to the `Route()` arg since route-rules bind bundles, not models.
- **Open: patent FTO** — A/B *metering/attribution* overlaps the FTO items flagged in AIQG-AGENT-IDENTITY-RESEARCH.md; clear before any billing use of experiment results (observability use is fine).

14. Work breakdown (per repo, Phase 2)

1. **aiqq-dashboard-be** — experiment tables + CRUD API + /results (experiment_variant rollup + Loki status) + /internal/experiments cache-load + tas.aiqq.experiment.changed.v1 emit + the **periodic auto-stop evaluator** (reads 1m rollup + Loki status, trips paused).
2. **tas-llm-router** — experiment resolver + per-tenant cache + assignment (reuse identity ladder) + Route() override arg + experiment_id/variant on the event + emitter promotion. The core, and the production-routing change — lands behind dry-run first.
3. **tas-spark-jobs** (Phase 2b, NOT deferred) — emit the scope_type='experiment_variant' row from aiqq-aggregator so per-variant metrics are pre-aggregated in aiqq.metrics_1m. Load-bearing for guardrails.
4. **aiqq-ui** — experiments list + create/edit wizard (objective + variant axis + quality metric + guardrails) + live results (per-variant comparison incl. quality/feedback, pause).
5. **aether-shared/data-models/aiqq/** — new experiment.md + response-feedback.md model docs + aggregated-metrics.md update for the new scope_type.

Shared quality capabilities (experiments *consume*, don't own — sequence independently):

6. **Explicit feedback** (§6.5) — POST /api/v1/feedback + aiqq.response_feedback table + correlation (response_event_id / client_request_id) + optional TAS-Response-Event-Id response header on the gateway. Useful platform-wide (also feeds Efficacy + the AI Quality report), so build standalone; the experiment results query joins/aggregates it per variant.
7. **LLM-as-judge** (§6.6) — the Phase-2 Efficacy augmentation: a judge service that rubric-scores a sample by workflow_type, plus the per-type automatic signals (code execution, schema/label match). Owned by the Efficacy/scoring track; experiments call into it (pointwise live, pairwise in shadow-eval). **An experiment with a quality objective is blocked until at least the structural tier + one of {feedback, judge} is available for its workflow type.**